

电子产品销售与利润分析及高价值订单预测

项目摘要

时间：2025.12-2026.03

项目描述

该项目对一个电子产品相关企业横跨三年的运营数据进行了预处理，并从时间、区域、产品三个维度进行了销售与利润上的分析，不仅识别了该企业在发展上的时间趋势与地区差异，还对该企业旗下产品组合进行了总结与比较；同时利用机器学习技术构造了不同的模型来预测较高利润订单，由此辅助企业运营与决策，促进企业发展；最终为企业挖掘出了有利于公司运营、销售规模稳定扩张与利润高质量增长的潜在优势。

数据概况

该数据集总共由3140条订单数据构成，每条数据共有7个属性：订单日期、所购产品名、分类、地区、销量、销售额与利润。初步检查后发现该数据集存在部分字段缺失与数据异常的情况，故先对其进行了数据清理；而后又根据分析目的不同从不同字段中构造提取出额外的有效数据。

主要结论与建议

- 1.该企业在三年内的销售规模持续扩大，但利润增长不稳定，盈利能力也在持续下滑，利润增长质量较差；该企业是否该考虑从追求销售规模的扩大转向关注利润质量的提高。
- 2.各地子公司对企业整体增长的贡献较为均匀，但个别地区存在即使经营规模扩大但盈利能力却持续下降的恶化趋势；该企业是否需要叫停该地区恶性扩张并对其进行部分审查。
- 3.该企业旗下产品种类丰富，可组合性较强，但不同产品对公司经营贡献差距较大，盈利效率也有一定差异；该企业可以考虑重新设计产品策略，促进各产品在企业发展中的稳定均衡贡献。
- 4.少数高销售额高利润订单贡献了超过两成的利润增长，同时表现出与普通订单不一样的产品结构 with 盈利模式；该企业或该深入研究大额订单的特征，以此辅助公司运营与销售策略相关决策。

目录

- 0.准备工作
- 1.数据预处理
 - 1.1数据概览
 - 1.2数据清理 - 空值, 重复值, 缺失值, 异常点, 离群点
 - 1.3特征工程(数据分析)
- 2.探索性数据分析(EDA)与数据可视化
 - 2.1趋势分析
 - 2.2区域分析
 - 2.3产品分析 - 产品组合战略矩阵
- 3.离群点分析 - 异常高销售额高利润订单
- 4.机器学习与较高利润订单预测
 - 4.1逻辑回归 - 第一次迭代
 - 4.1.1特征工程(机器学习))
 - 4.1.2模型定义与训练
 - 4.1.3性能检查与前K名精确率(Precision@K))
 - 4.1.4数据泄露与无效变量
 - 4.2逻辑回归 - 第二次迭代
 - 4.3随机森林
- 5.结论、感悟与展望

0.准备工作

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.ticker import PercentFormatter, FuncFormatter
import seaborn as sns
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.metrics import classification_report, confusion_matrix
from sklearn.metrics import roc_auc_score, average_precision_score
from sklearn.ensemble import RandomForestClassifier

sns.set_theme(style='darkgrid', palette='deep')

df = pd.read_csv('sales_data_electronic_devices.csv')
```

1. 数据预处理

1.1数据概览

```
In [2]: df.head()
```

Out [2]:

	Order Date	Product Name	Category	Region	Quantity	Sales	Profit
0	2022-11-27	Mouse	Accessories	East	3	598.665	53.265
1	2022-05-11	Tablet	Electronics	South	5	5865.570	502.730
2	2024-03-16	Mouse	Accessories	South	1	393.480	101.435
3	2022-09-10	Mouse	Accessories	West	0	254.590	51.640
4	2023-12-01	Camera	Electronics	West	1	524.380	106.350

```
In [3]: df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 3140 entries, 0 to 3139
Data columns (total 7 columns):
#   Column          Non-Null Count  Dtype
---  -
0   Order Date      3124 non-null   str
1   Product Name    3131 non-null   str
2   Category        3116 non-null   str
3   Region          3140 non-null   str
4   Quantity        3140 non-null   int64
5   Sales           3140 non-null   float64
6   Profit          3140 non-null   float64
dtypes: float64(2), int64(1), str(4)
memory usage: 171.8 KB
```

1.2 数据清理

在前期工作中，我发现数据集中存在不同类型的空值，例如NaN, "", ' ', #NAME等，所以我决定根据不同的列的具体情况采取不同的措施来处理。

```
In [4]: #订单日期列
#检查空值数量，确保删除所有空值

df['Order Date'] = df['Order Date'].str.strip()
df['Order Date'] = df['Order Date'].replace(['', 'NaN', 'na', 'Null'], pd.NA)

df['Order Date'].isna().sum()
```

```
Out[4]: np.int64(16)
```

```
In [5]: df = df.dropna(subset = 'Order Date').reset_index(drop = True)
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 3124 entries, 0 to 3123
Data columns (total 7 columns):
#   Column          Non-Null Count  Dtype
---  -
0   Order Date      3124 non-null   str
1   Product Name    3115 non-null   str
2   Category        3101 non-null   str
3   Region          3124 non-null   str
4   Quantity        3124 non-null   int64
5   Sales           3124 non-null   float64
6   Profit          3124 non-null   float64
dtypes: float64(2), int64(1), str(4)
memory usage: 171.0 KB
```

```
In [6]: #产品名称列

df['Product Name'].value_counts(dropna=False)
```

```
Out[6]: Product Name
Monitor      362
Camera       360
Smartwatch   360
Keyboard     348
Tablet       346
Mouse        341
Laptop       333
Smartphone   327
Headphones   225
Printer      106
NaN           9
Null          4
#NAME         3
Name: count, dtype: int64
```

```
In [7]: df['Product Name'] = df['Product Name'].replace(['#NAME', 'NaN', 'Null'], pd.NA)
df = df.dropna(subset = 'Product Name').reset_index(drop = True)
df['Product Name'].value_counts()
```

```
Out[7]: Product Name
Monitor      362
Camera       360
Smartwatch   360
Keyboard     348
Tablet       346
Mouse        341
Laptop       333
Smartphone   327
Headphones   225
Printer      106
Name: count, dtype: int64
```

```
In [8]: #类别列

df['Category'].value_counts(dropna=False)
```

```
Out[8]: Category
Electronics   1713
Accessories   1268
Office        105
NaN           14
na            4
Null          3
              1
Name: count, dtype: int64
```

```
In [9]: df['Category'] = df['Category'].str.strip()
df['Category'] = df['Category'].replace(['', 'NaN', 'na', 'Null'], pd.NA)

df = df.dropna(subset = 'Category').reset_index(drop = True)
df['Category'].value_counts()
```

```
Out[9]: Category
Electronics   1713
Accessories   1268
Office        105
Name: count, dtype: int64
```

In [10]: #区域列 (未发现空值)

```
df['Region'].value_counts(dropna=False)
```

Out[10]: Region
West 791
East 767
South 767
North 761
Name: count, dtype: int64

In [11]: #数值列, 查询描述性统计表

```
df.describe()
```

Out[11]:

	Quantity	Sales	Profit
count	3086.000000	3086.000000	3086.000000
mean	5.202852	3223.404543	555.817477
std	4.484478	3010.604611	639.363738
min	-1.000000	28.410000	-7662.670000
25%	2.000000	1027.622500	153.595000
50%	5.000000	2320.160000	357.900000
75%	7.000000	4535.687500	735.442500
max	168.000000	30470.070000	5676.899000

重点检查具有非正数Quantity的数据行以及非正数Profit的数据行。

In [12]: df['Quantity'].value_counts()

Out[12]: Quantity
2 398
4 380
3 350
1 337
6 300
8 294
5 286
7 277
9 246
0 41
12 39
16 37
18 36
10 31
14 31
-1 2
168 1
Name: count, dtype: int64

In [13]: df.loc[df['Quantity']<=0]

Out [13]:

	Order Date	Product Name	Category	Region	Quantity	Sales	Profit
3	2022-09-10	Mouse	Accessories	West	0	254.590	51.640
28	2023-02-07	Headphones	Accessories	South	-1	3252.370	713.460
127	2022-03-31	Keyboard	Accessories	South	0	94.640	7.440
209	2022-08-12	Mouse	Accessories	South	0	593.510	103.140
254	2023-04-30	Keyboard	Accessories	North	0	84.185	8.920
255	2024-11-01	Mouse	Accessories	South	0	529.230	103.775
295	2022-12-27	Keyboard	Accessories	North	0	81.985	11.035
337	2022-01-19	Mouse	Accessories	East	0	372.840	75.610
390	2023-10-02	Keyboard	Accessories	East	0	508.990	34.665
414	2023-06-04	Mouse	Accessories	North	0	325.930	93.995
419	2022-10-04	Mouse	Accessories	South	0	136.225	22.665
539	2022-08-03	Keyboard	Accessories	South	0	578.725	126.040
662	2023-12-01	Keyboard	Accessories	South	0	589.245	80.530
671	2023-03-22	Keyboard	Accessories	North	0	338.650	27.070
750	2023-06-07	Keyboard	Accessories	West	0	28.410	4.580
759	2023-04-02	Keyboard	Accessories	North	0	60.180	11.930
822	2022-06-14	Mouse	Accessories	North	0	369.760	55.590
867	2022-06-17	Keyboard	Accessories	South	0	83.290	15.985
1023	2023-05-10	Keyboard	Accessories	East	0	586.260	62.450
1119	2022-12-14	Monitor	Accessories	West	0	6968.010	1226.220
1135	2024-01-22	Keyboard	Accessories	West	0	193.185	57.450
1268	2023-02-16	Mouse	Accessories	East	0	424.945	102.940
1364	2023-08-02	Mouse	Accessories	East	0	516.045	145.705
1695	2023-11-11	Mouse	Accessories	North	0	229.850	62.595
1740	2024-12-30	Mouse	Accessories	North	0	373.305	65.625
1896	2022-12-05	Keyboard	Accessories	South	0	180.655	29.400
1913	2023-11-05	Mouse	Accessories	West	0	456.645	81.120
1971	2022-01-31	Mouse	Accessories	South	0	579.920	172.110
1992	2022-01-18	Keyboard	Accessories	East	0	358.800	68.635
2050	2023-08-16	Smartphone	Electronics	South	-1	2444.720	155.420
2053	2022-07-24	Keyboard	Accessories	North	0	434.325	30.060
2067	2022-02-19	Keyboard	Accessories	East	0	291.005	14.955
2245	2024-07-27	Keyboard	Accessories	East	0	353.410	31.910
2254	2022-10-19	Keyboard	Accessories	East	0	375.105	18.985

	Order Date	Product Name	Category	Region	Quantity	Sales	Profit
2293	2022-05-08	Mouse	Accessories	South	0	581.295	110.465
2337	2023-03-17	Keyboard	Accessories	North	0	473.580	38.605
2398	2022-01-29	Mouse	Accessories	West	0	516.680	151.340
2413	2023-03-04	Mouse	Accessories	West	0	156.265	20.235
2621	2022-07-31	Keyboard	Accessories	North	0	179.120	38.920
2725	2024-12-26	Mouse	Accessories	South	0	53.140	3.620
2781	2024-01-08	Keyboard	Accessories	East	0	155.810	25.330
2927	2022-03-18	Mouse	Accessories	West	0	52.085	14.835
3042	2022-12-15	Mouse	Accessories	North	0	282.320	82.250

Quantity为-1的订单只有两个，或为退货，但由于数量过于稀少，对分析来说不具有代表性，故直接删除。

Quantity为0的数据行可能由于销售人员的忘记录入或者数据损坏等等，显然为异常值，也直接删除。

In [14]:

```
mask = mask_neg = df['Quantity']<= 0
df = df[~mask_neg]
```

In [15]:

```
df['Quantity'].value_counts()
```

Out[15]:

Quantity	
2	398
4	380
3	350
1	337
6	300
8	294
5	286
7	277
9	246
12	39
16	37
18	36
10	31
14	31
168	1

Name: count, dtype: int64

In [16]:

```
df.loc[df['Profit']<=0]
```

Out[16]:

	Order Date	Product Name	Category	Region	Quantity	Sales	Profit
1771	2023-02-10	Camera	Electronics	West	7	5355.07	-2603.38
1986	2023-08-24	Keyboard	Accessories	South	4	3788.24	-7662.67

利润为负的异常点仅为两个，数量过于稀少，对后续分析也不具有代表性，故直接删除。

```
In [17]: #构造掩码
mask = mask_neg = df['Profit']<= 0
df = df[~mask_neg]
```

```
In [18]: # 检查是否存在重复值

df.duplicated().sum()
```

```
Out[18]: np.int64(0)
```

```
In [19]: #删除离群点

'''
1.遍历所有产品名
2.对每一种产品，利用IQR法，分别查找他们在Sales与Profit上的outlier，并构造index mask
3.删除
'''

def outlier_mask(serie: pd.Series):
    '''
    1.参数:
        s:某一种产品的所有数据点
    2.return: 构造出的index mask
    '''

    quantile1 = serie.quantile(0.25)
    quantile2 = serie.quantile(0.75)
    iqr = quantile2 - quantile1

    upperBound = quantile2 + 1.5*iqr
    lowerBound = quantile1 - 1.5*iqr

    mask = (serie>upperBound) | (serie < lowerBound)
    return mask

def drop_outlier_per_product(dataFrame:pd.DataFrame,outlier_index:list,outlier:
    '''
    1.参数
        dataFrame: 该表整体数据
        outlier_index: 找到的离群点在总体表中的index
        outlier: 容器，用于存放找到的离群点
    '''
    outlier_index = []

    for name in dataFrame['Product Name'].unique():

        mask = dataFrame['Product Name'] == name

        temp = dataFrame.loc[mask]['Sales']
        innormalSales = outlier_mask(temp)

        dataFrame.loc[mask]['Profit']
        innormalProfit = outlier_mask(dataFrame.loc[mask]['Profit'])

        outlier = innormalSales | innormalProfit

        outlier_index.extend(temp[outlier].index)

    outlier = dataFrame.loc[outlier_index]
```

```

dataFrame = dataFrame.drop(index=outlier_index).reset_index(drop = True)

return dataFrame,outlier_index,outlier

outlier_index = []
outlier = None

data_before_outlied = df.copy() #备份, 以便于后续离群点分析 :)

df,outlier_index,outlier = drop_outlier_per_product(df,outlier_index,outlier)
df.info()

```

```

<class 'pandas.DataFrame'>
RangeIndex: 2834 entries, 0 to 2833
Data columns (total 7 columns):
#   Column          Non-Null Count  Dtype
---  -
0   Order Date      2834 non-null   str
1   Product Name    2834 non-null   str
2   Category        2834 non-null   str
3   Region          2834 non-null   str
4   Quantity        2834 non-null   int64
5   Sales           2834 non-null   float64
6   Profit          2834 non-null   float64
dtypes: float64(2), int64(1), str(4)
memory usage: 155.1 KB

```

1.3特征工程(数据分析)

对数据的部分列进行一些类型转换, 将其从String类型转换为对分析更有效的类型; 其次再从已有数据列中提取出一些额外的数据列, 用以辅助后续的数据分析工作。

In [20]: #数据列类型转换: 将订单日期转换为pandas内置的Datetime类型, 将产品名、类别与区域三个String类

```

df['Order Date'] = pd.to_datetime(df['Order Date'])
df['Product Name'] = df['Product Name'].astype('category')
df['Category'] = df['Category'].astype('category')
df['Region'] = df['Region'].astype('category')

df.info()

```

```

<class 'pandas.DataFrame'>
RangeIndex: 2834 entries, 0 to 2833
Data columns (total 7 columns):
#   Column          Non-Null Count  Dtype
---  -
0   Order Date      2834 non-null   datetime64[us]
1   Product Name    2834 non-null   category
2   Category        2834 non-null   category
3   Region          2834 non-null   category
4   Quantity        2834 non-null   int64
5   Sales           2834 non-null   float64
6   Profit          2834 non-null   float64
dtypes: category(3), datetime64[us](1), float64(2), int64(1)
memory usage: 97.1 KB

```

In [21]: #构造单价与销售利润率两个新的数据列

```
df['Unit Price'] = df['Sales']/df['Quantity']
df['Profit Margin'] = df['Profit']/df['Sales']
df.head()
```

Out [21]:

	Order Date	Product Name	Category	Region	Quantity	Sales	Profit	Unit Price	
0	2022-11-27	Mouse	Accessories	East	3	598.665	53.265	199.555000	0.0
1	2022-05-11	Tablet	Electronics	South	5	5865.570	502.730	1173.114000	0.0
2	2024-03-16	Mouse	Accessories	South	1	393.480	101.435	393.480000	0.0
3	2023-12-01	Camera	Electronics	West	1	524.380	106.350	524.380000	0.0
4	2023-10-09	Headphones	Accessories	North	7	6167.320	1027.980	881.045714	0.0

In [22]: #添加 年, 年-月, 季度三个新列并重新排列各数据列顺序

```
df['Year'] = df['Order Date'].dt.to_period('Y')
df['Year-Month'] = df['Order Date'].dt.to_period('M')
df['Quarter'] = df['Order Date'].dt.to_period('Q')

df = df[['Year', 'Quarter', 'Year-Month', 'Order Date', 'Region', 'Category', 'Product Name', 'Unit Price', 'Quantity', 'Sales', 'Profit', 'Profit Margin']]
df.info()
```

<class 'pandas.DataFrame'>

RangeIndex: 2834 entries, 0 to 2833

Data columns (total 12 columns):

#	Column	Non-Null Count	Dtype
0	Year	2834 non-null	period[Y-DEC]
1	Quarter	2834 non-null	period[Q-DEC]
2	Year-Month	2834 non-null	period[M]
3	Order Date	2834 non-null	datetime64[us]
4	Region	2834 non-null	category
5	Category	2834 non-null	category
6	Product Name	2834 non-null	category
7	Unit Price	2834 non-null	float64
8	Quantity	2834 non-null	int64
9	Sales	2834 non-null	float64
10	Profit	2834 non-null	float64
11	Profit Margin	2834 non-null	float64

dtypes: category(3), datetime64[us](1), float64(4), int64(1), period[M](1), period[Q-DEC](1), period[Y-DEC](1)

memory usage: 207.8 KB

In [23]: # 备份 (是个好习惯):)

```
df.to_csv('cleaned_padded_data.csv')
```

2.探索性数据分析与可视化

2.1趋势分析

In [24]:

```
# 查询三年总销售额与总利润；计算三年总销售利润率

overall_summary = df[['Sales', 'Profit']].sum()
overall_summary = pd.DataFrame(overall_summary).T
overall_summary['Profit Margin'] = overall_summary['Profit']/overall_summary['Sales']
overall_summary
```

Out [24]:

	Sales	Profit	Profit Margin
0	7941206.13	1304317.46	0.164247

In [25]:

```
#查询每年销售额与利润；计算每年销售利润率

annual_summary = df.groupby('Year').agg({'Sales': 'sum', 'Profit': 'sum'})
annual_summary['Profit Margin'] = annual_summary['Profit']/annual_summary['Sales']
annual_summary = annual_summary.reset_index()
annual_summary
```

Out [25]:

	Year	Sales	Profit	Profit Margin
0	2022	2441843.550	411430.770	0.168492
1	2023	2721502.895	451579.225	0.165930
2	2024	2777859.685	441307.465	0.158866

In [26]:

```
#数据可视化： 分别构造三张图显示销售额， 利润， 利润率以及他们在三年间的变化

temp = annual_summary.copy()
temp['Year'] = temp['Year'].dt.year

fig, axes = plt.subplots(1, 3, figsize=(18, 5))

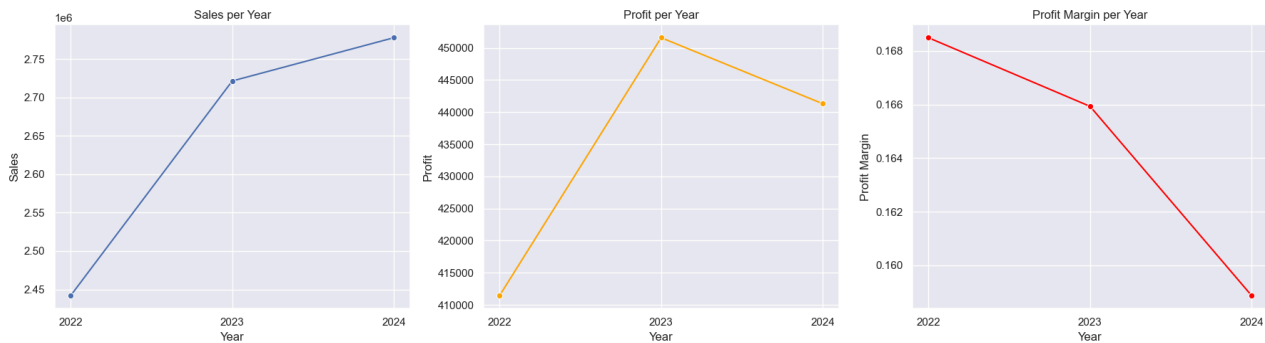
#销售额
sns.lineplot(data=temp, x='Year', y='Sales', marker='o', ax=axes[0])
axes[0].set_title('Sales per Year')
axes[0].set_xlabel('Year')
axes[0].set_ylabel('Sales')
axes[0].set_xticks(temp['Year'])

#利润
sns.lineplot(data=temp, x='Year', y='Profit', marker='o', ax=axes[1], color='orange')
axes[1].set_title('Profit per Year')
axes[1].set_xlabel('Year')
axes[1].set_ylabel('Profit')
axes[1].set_xticks(temp['Year'])

#利润率
sns.lineplot(data=temp, x='Year', y='Profit Margin', marker='o', ax=axes[2], color='green')
```

```
axes[2].set_title('Profit Margin per Year')
axes[2].set_xlabel('Year')
axes[2].set_ylabel('Profit Margin')
axes[2].set_xticks(temp['Year'])

plt.tight_layout()
plt.show()
```



In [27]: #计算销售额、利润逐年增长率；计算销售利润率逐年变化

```
annual_summary['Sales Growth Rate'] = annual_summary['Sales'].pct_change()
annual_summary['Profit Growth Rate'] = annual_summary['Profit'].pct_change()
annual_summary['Profit Margin Change'] = annual_summary['Profit Margin'] - annu

#计算销售额、利润两年总体增长率；计算销售利润率两年总体变化
annual_summary['Sales Growth Rate per 2 Year'] = pd.Series([np.nan,np.nan,(annu
annual_summary['Profit Growth Rate per 2 Year'] = pd.Series([np.nan,np.nan,(ann
annual_summary['Profit Margin Change per 2 Year'] = pd.Series([np.nan,np.nan,(a

#重排序
annual_summary = annual_summary[['Year','Sales','Sales Growth Rate','Sales Grow
annual_summary
```

Out [27]:

	Year	Sales	Sales Growth Rate	Sales Growth Rate per 2 Year	Profit	Profit Growth Rate	Profit Growth Rate per 2 Year	Profit Margin	
0	2022	2441843.550	NaN	NaN	411430.770	NaN	NaN	0.168492	
1	2023	2721502.895	0.114528	NaN	451579.225	0.097583	NaN	0.165930	-0
2	2024	2777859.685	0.020708	0.137608	441307.465	-0.022746	0.072617	0.158866	-0

从表格与图像中可以看出，2022年至2024年三年间，该公司销售额总计约7941206元（约合794万），利润总计约1304317元（约合130万），三年总销售利润率约为16.42%；平均每100元销售额可获利16.42元。同时图像也显示，三年间该公司销售额逐年上升，三年总计增长率约13.76%；2022至2023年为11.45%，2024年则放缓为约2%。在利润方面，虽然该公司三年内总利润从大约411430元上升至大约441307元，三年总增长率约为7.26%，但可以看到该公司只在2022到2023年实现了利润增长，而且也达到了三年中的利润峰值约451579（约合45万元），增长率为9.76%；其后在2024年下降至约441307元（约合44万元），降幅约2.27%。

值得注意的是，虽然该公司的销售额在三年间持续增长，但利润在2024年已开始回落，而且每年销售利润率也在连续下降。2022年销售利润率大约为16.85%，到2024年已下降到15.89%。下跌接近1个百分点，呈现出明显的跌落趋势。

总体来说，该公司在三年时间里销售规模持续扩大，虽然扩张势头良好，但利润提升却不稳定，增长质量也不牢固，而且盈利能力逐年下滑。我建议该公司是否考虑减缓扩张速度，将重心转到提高利润与销售利润率上，逐步强化自身盈利能力，促进公司高质量稳定发展。

2.2区域分析

In [28]:

```
# 计算各地区三年总销售额

sales_each_region = df.groupby('Region').agg({'Sales':'sum'}).reset_index()
sales_each_region
```

Out [28]:

	Region	Sales
0	East	2037223.585
1	North	1853480.965
2	South	1940823.735
3	West	2109677.845

In [29]:

```
# 计算各地区三年总利润

profit_each_region = df.groupby('Region').agg({'Profit':'sum'}).reset_index()
profit_each_region
```

Out [29]:

	Region	Profit
0	East	333456.575
1	North	306190.135
2	South	318092.145
3	West	346578.605

In [30]:

```
# 计算各地区三年总利润率并构造各地区三年总结

annual_summary_each_region = pd.merge(sales_each_region,profit_each_region,on =
annual_summary_each_region['Profit Margin'] = annual_summary_each_region['Profi
annual_summary_each_region
```

Out [30]:

	Region	Sales	Profit	Profit Margin
0	East	2037223.585	333456.575	0.163682
1	North	1853480.965	306190.135	0.165197
2	South	1940823.735	318092.145	0.163895
3	West	2109677.845	346578.605	0.164280

In [31]:

```
#数据可视化： 分别显示各地区销售额， 利润及其利润占比

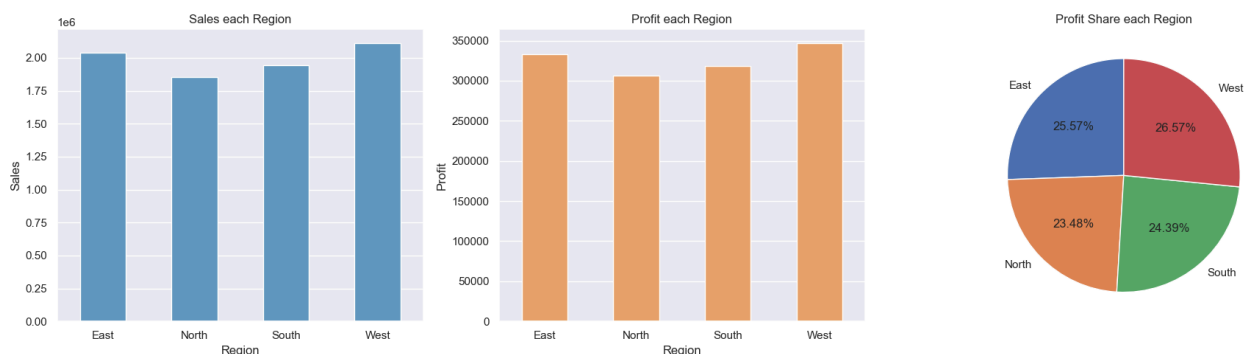
temp = annual_summary_each_region.copy()
fig, axes = plt.subplots(1, 3, figsize=(18, 5))
```

```
#各地区销售额
sns.barplot(data=temp, x='Region', y='Sales', ax=axes[0],width=0.5,color = sns.
axes[0].set_title('Sales each Region')
axes[0].set_xlabel('Region')
axes[0].set_ylabel('Sales')

#各地区利润
sns.barplot(data=temp, x='Region', y='Profit', ax=axes[1],color = sns.color_pal
axes[1].set_title('Profit each Region')
axes[1].set_xlabel('Region')
axes[1].set_ylabel('Profit')

#各地区利润占比
axes[2].pie(
    temp['Profit'],
    labels=temp['Region'],
    autopct='%.2f%%',
    startangle=90)
axes[2].set_title('Profit Share each Region')

plt.tight_layout()
plt.show()
```



由数据与图表可以得出，四个地区中三年总销售额与总利润最高的均为西部地区，二者分别为2109677（约合210万元）与346578（约合34万元），同时对三年总利润的贡献也以26.57%居第一位；其余三地贡献则较为均匀，但与西部地区仍有1到3个百分点的差距。销售利润率方面，四地的三年总销售利润率均为16%上下，差距未超过1个百分点。可以看出，在2022到2024三年间，不管是销售额还是利润，各地区之间虽然存在微小差异，但对公司整体贡献依然比较均匀；各地区销售利润率差距未超过1个百分点，各地区盈利能力不相上下。

In [32]: # 查询各地区逐年销售额与逐年利润，计算逐年销售利润率，构造逐年总结

```
annual_summary_each_region_per_year = df.groupby(['Region','Year']).agg({'Sales
annual_summary_each_region_per_year['Profit Margin'] = annual_summary_each_regi
annual_summary_each_region_per_year
```

Out [32]:

		Sales	Profit	Profit Margin
Region	Year			
East	2022	637508.790	110437.760	0.173233
	2023	699725.395	114168.365	0.163162
	2024	699989.400	108850.450	0.155503
North	2022	610328.805	96847.875	0.158681
	2023	633366.815	105750.360	0.166965
	2024	609785.345	103591.900	0.169883
South	2022	608748.110	107030.525	0.175821
	2023	657786.080	111848.685	0.170038
	2024	674289.545	99212.935	0.147137
West	2022	585257.845	97114.610	0.165935
	2023	730624.605	119811.815	0.163985
	2024	793795.395	129652.180	0.163332

In [33]: #可视化: 三张图分别显示 1. 各地区逐年销售额 2. 各地区逐年利润 3. 各地区逐年利润率

```
temp = annual_summary_each_region_per_year.reset_index()
temp['Year'] = temp['Year'].dt.year

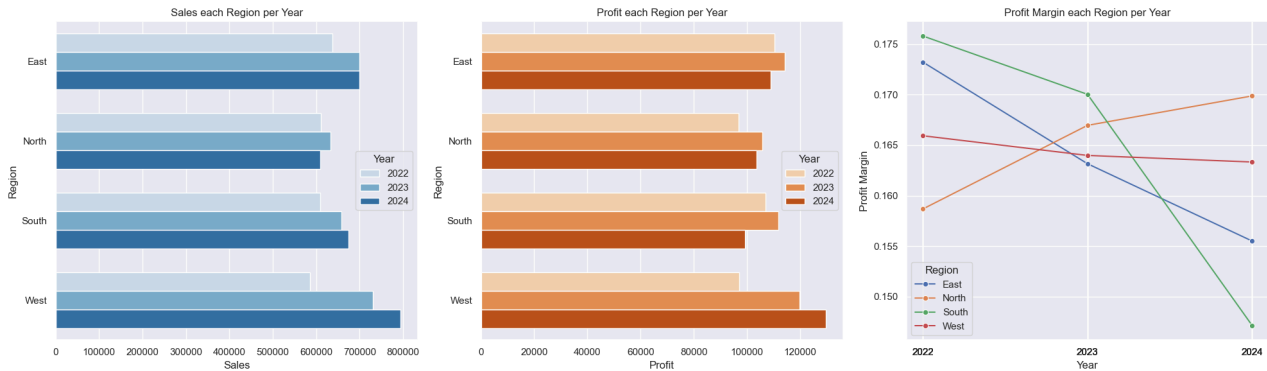
fig, axes = plt.subplots(1, 3, figsize=(20, 6))
year_palette = sns.color_palette('Blues', 3)

#各地区逐年销售额
sns.barplot(data=temp, y='Region', x='Sales', hue='Year', ax=axes[0], palette=year_p
axes[0].set_title('Sales each Region per Year')
axes[0].set_xlabel('Sales')
axes[0].set_ylabel('Region')
axes[0].legend(title='Year')

#各地区逐年利润
sns.barplot(data=temp, y='Region', x='Profit', hue='Year', ax=axes[1], palette=sns.c
axes[1].set_title('Profit each Region per Year')
axes[1].set_xlabel('Profit')
axes[1].set_ylabel('Region')
axes[1].legend(title='Year')

#各地区逐年销售利润率
sns.lineplot(data=temp, x='Year', y='Profit Margin', hue='Region', marker='o', ax=ax
axes[2].set_title('Profit Margin each Region per Year')
axes[2].set_xlabel('Year')
axes[2].set_ylabel('Profit Margin')
axes[2].legend(title='Region')
axes[2].set_xticks(temp['Year'])

plt.tight_layout()
plt.show()
```



销售额方面，东部、南部与西部地区均在三年间实现了连续增长，其中西部与南部地区整体增长较大，截止到2024年结束分别取得了793795（约合79万元）与674289（约合67万），位列第一第二；东部地区则只在2023年实现了一定突破，在2024年则仅有微小的增长；北地区则在短暂的销售额增长过后又开始下降，到2024年销售规模已经萎缩到略低于2022年的水平。利润方面，唯一实现三年利润连续增长的只有西部地区，其余三地均只在中间一年实现了一定增长，而后又开始回落，其中东部地区与南部地区甚至在2024年萎缩到低于2022年的利润水平。

同时值得注意的是，虽然北部地区利润在2024年有一定回落，但销售利润率却从2022年的15.87%逐年提升至2024年的16.99%，涨幅超一个点，利润虽有下降，但增长质量实现了逐年提高；另一方面，尽管西部地区的销售额与利润在三年间均为持续增长，但销售利润率却从2022年的16.59%持续下跌至2024年的16.33%，盈利效率持续下跌。该地区分公司是否考虑减缓扩张速度，转而提升盈利效率，为以后的高质量发展奠定坚实基础。

2.3产品分析与产品组合战略矩阵

```
In [34]: #查询三年间各产品的销售额、利润，并计算各产品的总体销售利润率

summary_each_product = df.groupby('Product Name').agg({'Sales':'sum','Profit':'sum'})
summary_each_product['Profit Margin'] = summary_each_product['Profit'] / summary_each_product['Sales']
```

Out [34]:

	Product Name	Sales	Profit	Profit Margin
0	Camera	1020240.540	168956.93	0.165605
1	Headphones	573377.880	96090.35	0.167586
2	Keyboard	551225.655	86887.87	0.157627
3	Laptop	1125867.960	200415.28	0.178010
4	Monitor	958829.840	153867.38	0.160474
5	Mouse	627848.525	103792.13	0.165314
6	Printer	252306.750	36531.43	0.144790
7	Smartphone	1195775.600	199463.02	0.166806
8	Smartwatch	841254.900	132862.17	0.157933
9	Tablet	794478.480	125450.90	0.157903

```
In [35]: #可视化：构造三张图表分别表示 1. 各产品总体销售额 2. 各产品总体利润 以及 3. 各产品总体销售利润率
```

```

temp = summary_each_product.reset_index()
fig, axes = plt.subplots(1, 3, figsize=(20, 6))

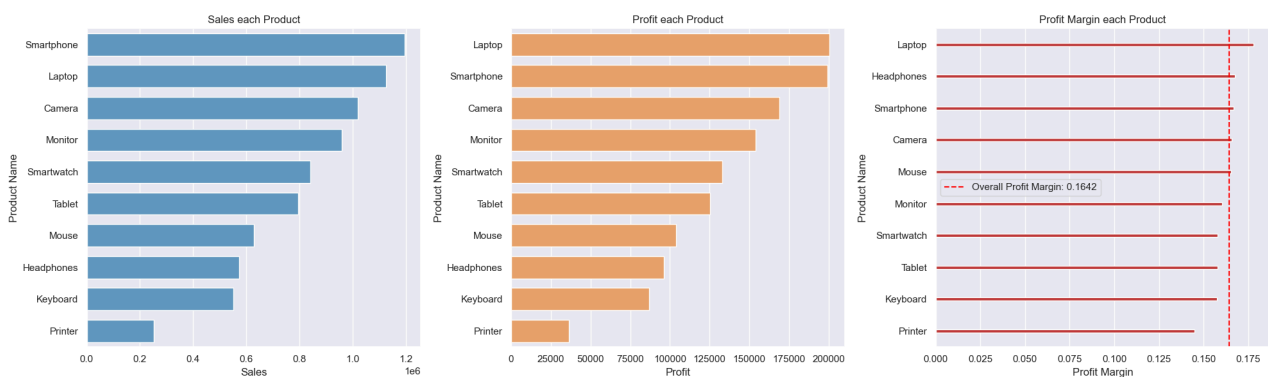
#各产品总体销售额
sns.barplot(data=temp,y='Product Name',x='Sales',ax=axes[0],color=sns.color_pal
axes[0].set_title('Sales each Product')
axes[0].set_xlabel('Sales')
axes[0].set_ylabel('Product Name')

#各产品总体利润
sns.barplot(data=temp,y='Product Name',x='Profit',ax=axes[1],color=sns.color_pa
axes[1].set_title('Profit each Product')
axes[1].set_xlabel('Profit')
axes[1].set_ylabel('Product Name')

overall_profit_margin = overall_summary['Profit Margin'].iloc[0]
# 各地区逐年销售利润率
sns.barplot(data=temp,y='Product Name',x='Profit Margin',ax=axes[2],color=sns.c
axes[2].axvline(x=overall_profit_margin,color='red',linestyle='--',label=f"Over
axes[2].set_title('Profit Margin each Product')
axes[2].set_xlabel('Profit Margin')
axes[2].set_ylabel('Product Name')
axes[2].legend()

plt.tight_layout()
plt.show()

```



该公司总共销售十款产品，范围覆盖电子产品、配件以及部分办公用品。其中销售额最高的三名依次为智能手机、笔记本电脑和相机，三者均突破一百万元销售额大关，其销售额依次为1195775（约合119万元）、1125867（约合112万元）与1020240（约合102万元）。

利润方面前三名依次是笔记本电脑、智能手机与相机，利润分别为200415（约合20万元）、199463（约合19万元）与168956（约合16万元）；销售利润率方面，该公司只有一半产品微弱突破总的销售利润率16.4%，其中最高的为笔记本电脑，约为17.80%，只比三年总销售利润率高出约1.4个百分点，其余四项从高到低依次为头戴式耳机，16.76%，智能手机，16.68%，相机，16.56%以及鼠标16.53%。相机在销售额与利润方面均位列第三，但盈利能力由于头戴式耳机被挤到第四。

而在显示器、智能手表、平板电脑、鼠标、键盘与打印机这些产品中，不管是销售额还是利润亦或利润率均在中间偏下水平，其中销售额、利润与利润率最低的均为打印机，数值依次为252306（约合25万元）、36531（约合3.6万元）与14.48%，销售额最高的为显示器的958829（约合95万元），利润最高的为显示器的153867（约合15万元），利润率则为鼠标的最高，16.53%。

```
In [36]: #查询各产品销售额占比
overall_sales = overall_summary['Sales'].iloc[0]
summary_each_product['Sales Share'] = summary_each_product['Sales']/overall_sales
summary_each_product = summary_each_product[['Product Name','Sales','Sales Share']]
summary_each_product
```

```
Out [36]:
```

	Product Name	Sales	Sales Share	Profit	Profit Margin
0	Camera	1020240.540	0.128474	168956.93	0.165605
1	Headphones	573377.880	0.072203	96090.35	0.167586
2	Keyboard	551225.655	0.069413	86887.87	0.157627
3	Laptop	1125867.960	0.141775	200415.28	0.178010
4	Monitor	958829.840	0.120741	153867.38	0.160474
5	Mouse	627848.525	0.079062	103792.13	0.165314
6	Printer	252306.750	0.031772	36531.43	0.144790
7	Smartphone	1195775.600	0.150579	199463.02	0.166806
8	Smartwatch	841254.900	0.105935	132862.17	0.157933
9	Tablet	794478.480	0.100045	125450.90	0.157903

```
In [37]: #可视化: 构造产品组合战略矩阵 - 散点图

temp =summary_each_product.reset_index()
fig, ax = plt.subplots(figsize=(7, 6))

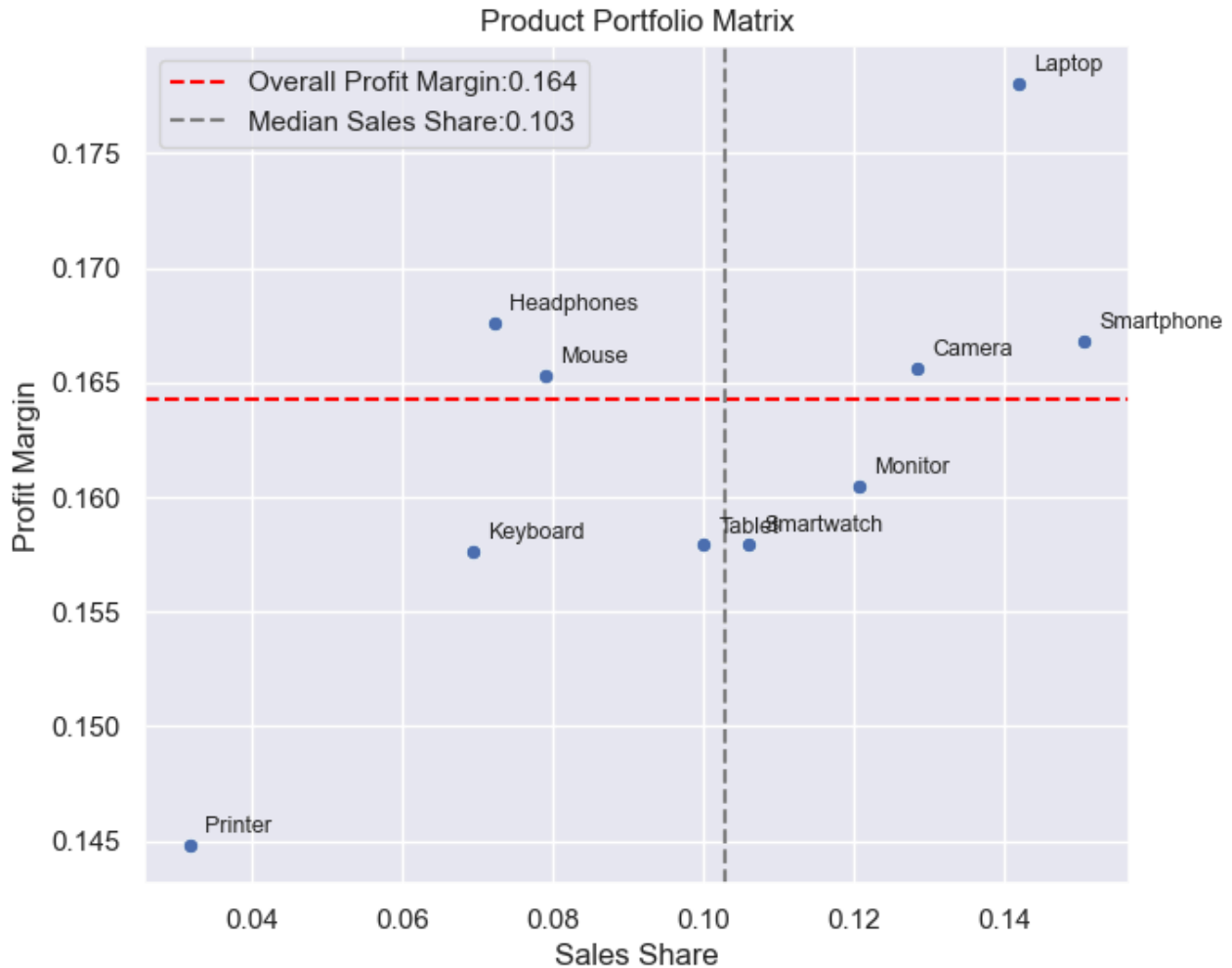
sns.scatterplot(data=summary_each_product,x='Sales Share',y='Profit Margin',ax=

#显示每个点对应的产品名字
for _, row in temp.iterrows():
    ax.annotate(row['Product Name'],xy=(row['Sales Share'], row['Profit Margin']
        xytext=(6, 5),textcoords='offset points',fontsize=9)

#刻画总体销售利润率
ax.axhline(y=overall_profit_margin,color='red',linestyle='--',label=f'Overall P

#刻画个各产品中的销售贡献中位数
sales_share_median = temp['Sales Share'].median()
ax.axvline(x=sales_share_median,color='gray',linestyle='--',label=f'Median Sale

ax.legend()
plt.title('Product Portfolio Matrix')
plt.show()
```



结合该产品组合矩阵与之前的三个指标图，可以确定该公司高销售额贡献与高利润率的核心产品为笔记本电脑、智能手机与相机；其中笔记本电脑盈利效率最高，平均每100元销售额可获利17.80元；智能手机的销售规模最大，三年累计销售额达约119万元。该公司是否考虑优先加大对此二者的资源投入，保持甚至提高他们作为该公司的核心产品的稳定优势定位，促进他们持续为公司贡献高占比、高质量的销售收入与利润增长。同时对相机来说，该公司是否考虑将其纳入后备核心产品范围内经营，逐步提高它成为公司核心产品的潜力与市场竞争力，推动他的销售额与利润的稳定增长。

其次，对头戴式耳机与鼠标这一类潜力产品来说，虽然他们对销售额的贡献不高，但盈利能力却不低；该公司是否考虑加大对这一类产品的营销投入与推广力度，一方面提升该类产品在潜在用户中的知名度，另一方面提高该类产品在目标人群中的曝光度，进而逐步提升他们在销售额中的占比，促进公司利润的高质量增长。与之相反的显示器、智能手表与平板电脑的这一类流量产品，则表现出较高销售额占比与较低盈利能力的态势；该公司是否考虑在一定程度上提高对这一类产品的投入，维持该类产品作为流量产品的定位，同时对其进行一定程度上的捆绑销售，争取在保持引流作用的前提下实现客单总利润的最大化。

最后对低销售额贡献与低销售利润率的问题产品来说，也即键盘与打印机，其中键盘在两个维度上的情况都比打印机略好，该公司是否考虑投入一定的资源对其进行部分的产品优化，亦或是考虑对其进行一定程度上的促销或者捆绑销售，逐步提升它在销售占比中的贡献与盈利能力。而对打印机来说，该公司是否考虑暂停该产品的销售，重新对该产品进行审核以及重做市场调研，必要时是否考虑放弃该产品，专攻电子产品市场，同时将该产品所占资源让渡给其他产品，促进其他产品在销售规模与利润提升的稳定高质量发展。

总的来说，该公司主要经营面为电子产品市场，产品类型丰富，可组合性较强；公司销售规模与利润增长的来源较广，构成丰富，而且未出现依赖单一产品的极端情况。但各产品盈利能力参差不齐，对销售规模与利润增长的贡献也存在一定差异，同时还有部分产品的销售规模与盈利能力的表现都较弱的情况存在。

3.离群点分析-异常高销售额高利润订单

之前的分析都是在剔除了离群点的数据集上进行的，这些记录虽然在统计意义上是离群点，但其实如果人们仔细想想就会发现，实际中的离群点不一定是异常点，他也可能是真实存在的高销售额高利润的订单。这样的大额订单对一家公司来说就有极大的意义与商业价值，同时这样的大额订单也会影响一家公司的资源投入与商业决策。

In [38]:

```
#查看离群值整体状况

outlier = outlier.reset_index(drop = True)
outlier.head()
```

Out [38]:

	Order Date	Product Name	Category	Region	Quantity	Sales	Profit
0	2023-12-25	Mouse	Accessories	North	6	7146.65	1627.42
1	2024-06-29	Mouse	Accessories	West	9	7506.65	594.28
2	2022-02-19	Mouse	Accessories	West	6	5622.74	1382.67
3	2023-08-12	Mouse	Accessories	South	6	6228.01	1852.45
4	2022-07-26	Mouse	Accessories	East	8	6056.23	1631.71

In [39]:

```
# 检查离群点数值分布情况

outlier.describe()
```

Out [39]:

	Quantity	Sales	Profit
count	207.000000	207.000000	207.000000
mean	9.541063	9524.530169	2013.729826
std	3.872608	4173.384328	974.796701
min	1.000000	3839.770000	448.620000
25%	7.000000	6798.840000	1414.380000
50%	9.000000	8240.650000	1777.320000
75%	9.000000	10363.995000	2329.800000
max	18.000000	30470.070000	5676.899000

In [40]:

```
#查询离群点总销售额

outlier_sales = outlier['Sales'].sum()
outlier_sales
```

Out [40]:

np.float64(1971577.7450000003)

In [41]:

```
#查询离群点总利润
```

```
outlier_profit = outlier['Profit'].sum()
outlier_profit
```

Out [41]: np.float64(416842.074)

In [42]: #计算离群点对总销售额与总利润贡献

```
sales_contribution = outlier_sales / data_before_outlied['Sales'].sum()
profit_contribution = outlier_profit / data_before_outlied['Profit'].sum()

outlier_contribution = pd.DataFrame({'Sales Contribution': [sales_contribution],
outlier_contribution
```

Out [42]:

	Sales Contribution	Profit Contribution
0	0.198892	0.242187

可以看到占比不到 207/3041 $\approx$ 6.8%的离群点贡献了高达19.89%的销售额与24.21%的利润，这对一家公司来说就是相当大的商机。

In [43]: #查看离群点按地区分布情况

```
outlier_each_region = outlier.groupby('Region').agg({'Sales': 'sum', 'Profit': 'sum'})
outlier_each_region
```

Out [43]:

	Region	Sales	Profit
0	East	452910.260	97770.800
1	North	523797.345	108510.254
2	South	426418.490	90293.140
3	West	568451.650	120267.880

In [44]: #可视化：构造三张图分别表示 1. 各区域离群点销售额 2. 各区域离群点利润 3. 西部离群点各产品利润
temp = outlier_each_region

```
fig, axes = plt.subplots(1, 3, figsize=(14, 4))
```

#各区域离群点销售额

```
sns.barplot(data=outlier_each_region, x='Sales', y='Region', ax=axes[0], color=sns.pastel1)
axes[0].set_title('Outlier Sales each Region')
axes[0].set_xlabel('Sales')
axes[0].set_ylabel('Region')
```

#各区域离群点利润

```
sns.barplot(data=outlier_each_region, x='Profit', y='Region', ax=axes[1], color=sns.pastel1)
axes[1].set_title('Outlier Profit each Region')
axes[1].set_xlabel('Profit')
axes[1].set_ylabel('Region')
```

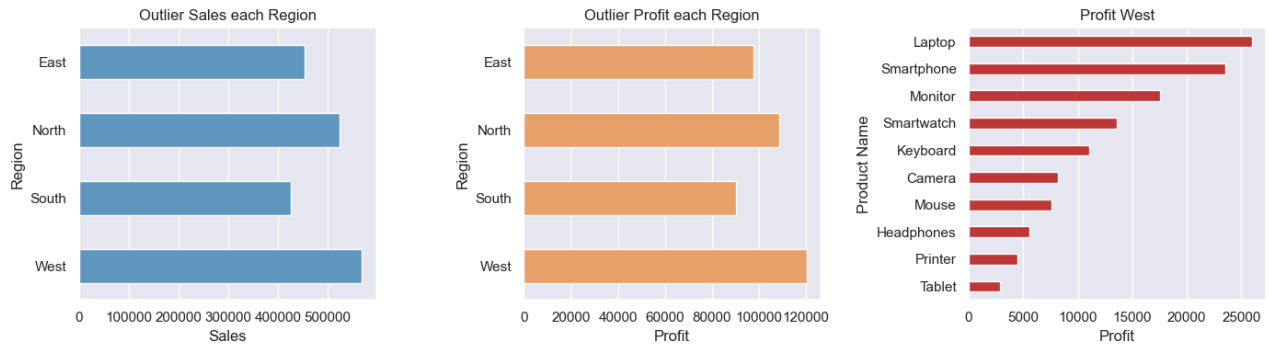
#西部离群点各产品利润

```
outlier_west = outlier.loc[outlier['Region'] == 'West']
outlier_west_each_product = outlier_west.groupby('Product Name').agg({'Sales': 'sum'})
outlier_west_each_product = outlier_west_each_product.reset_index()

sns.barplot(data=outlier_west_each_product, y='Product Name', x='Sales', ax=axes[2], color=sns.pastel1)
```

```
axes[2].set_title('Profit West')
axes[2].set_xlabel('Profit')
axes[2].set_ylabel('Product Name')

plt.tight_layout()
plt.show()
```



可以看出，在所有离群点中西部的销售额与利润均为第一，其数值依次为568451（约合56万元）与120267（约合12万元）；其中贡献利润的前三名依次是笔记本电脑，智能手机与显示器。笔记本电脑与智能手机作为该公司的核心产品在利润贡献上依旧具有领先地位；而令人诧异的是，作为公司流量产品的显示器、智能手表在利润贡献的表现上居然超过了作为核心产品的相机；甚至是被视为问题产品的键盘，在西部地区的利润贡献上也比相机要大得多。

由此可见，大额订单的利润构成与销售贡献分配与总体态势相比呈现出了一种完全不同的模式；该公司是否考虑以西部地区为例，深入研究大额订单的销售模式与盈利模式，认真探讨这类订单的产品组合构成与销售策略，加大对大额订单一定程度上的资源倾斜与销售投入，促进大额订单为该公司持续提供稳定的高质量的发展动力。

4.机器学习与较高利润订单预测

之前我们研究具有高销售额与高利润的离群点订单，并发现了这样的订单对该公司发展影响巨大，例如大额订单的利润贡献占比甚至超过两成；所以如果该公司可以提前预知某一订单是否为大额订单的话，或者是否有较高概率为大额订单的话，就可以对这样的订单进行实时追踪以及定向投入资源，例如销售人力，促进该类订单的完成率并由此提升利润增长。

基于此目的，现考虑将利润高于第75分位的订单定义为较高利润订单，并基于已有数据构造机器学习模型预测一个订单是否为较高利润订单，或者说，预测该订单是否为较高利润订单的概率，由此辅助公司进行动态商业决策，争取促成更多的高销售额高利润订单的完成。

4.1 逻辑回归

由于要预测的是概率高低，我们可以先考虑逻辑回归模型。该模型的输出介于[0,1]之间，我们可以将其解释为某一订单为较高利润订单的概率，并在模型训练完成以后，测试不同分类阈值对预测性能的影响，从中选出最好的作为分类阈值。

4.1.1特征工程(机器学习)

```
In [45]: data_lr = data_before_outlied.copy()
data_lr = data_lr.reset_index(drop = True)

# 将Order date分解为 年, 年-月, 季度, 日, 周内标识, 五个新列, 并添加 单价 一个新列
data_lr['Order Date'] = pd.to_datetime(data_lr['Order Date'])

data_lr['Year'] = data_lr['Order Date'].dt.year.astype('int32')
data_lr['Quarter'] = data_lr['Order Date'].dt.quarter.astype('int32')
data_lr['Month'] = data_lr['Order Date'].dt.month.astype('int32')
data_lr['Day'] = data_lr['Order Date'].dt.day.astype('int32')
data_lr['Weekday'] = data_lr['Order Date'].dt.dayofweek.astype('int32')

data_lr['Unit Price'] = data_lr['Sales']/data_lr['Quantity']

#选择必要的列并重新排列顺序
data_lr = data_lr[['Year', 'Quarter', 'Month', 'Day', 'Weekday', 'Region', 'Category']]

#转换产品名、地区与类别为分类类型
data_lr['Product Name'] = data_lr['Product Name'].astype('category')
data_lr['Region'] = data_lr['Region'].astype('category')
data_lr['Category'] = data_lr['Category'].astype('category')

data_lr.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 3041 entries, 0 to 3040
Data columns (total 12 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Year                   3041 non-null  int32
1   Quarter                3041 non-null  int32
2   Month                  3041 non-null  int32
3   Day                    3041 non-null  int32
4   Weekday                3041 non-null  int32
5   Region                 3041 non-null  category
6   Category               3041 non-null  category
7   Product Name           3041 non-null  category
8   Quantity               3041 non-null  int64
9   Unit Price             3041 non-null  float64
10  Sales                  3041 non-null  float64
11  Profit                 3041 non-null  float64
dtypes: category(3), float64(3), int32(5), int64(1)
memory usage: 163.6 KB
```

```
In [46]: #将利润大于前75%的订单标记为较高利润订单

bound = data_lr['Profit'].quantile(0.75)
y = (data_lr['Profit'] >= bound).astype(int)
X = data_lr[['Year', 'Quarter', 'Month', 'Day', 'Weekday', 'Region', 'Category', 'Prod
```

4.1.2模型定义与训练

```
In [47]: #编码各类数据并定义模型

cate_feature = ['Region', 'Category', 'Product Name']
num_feature = ['Year', 'Quarter', 'Month', 'Day', 'Weekday', 'Quantity', 'Unit P

preprocess = ColumnTransformer(transformers=[('cat', OneHotEncoder(), cate_fea

model = Pipeline(steps=[('preprocess', preprocess),
                        ('clf', LogisticRegression(max_iter=1000, class_weight='balance
```

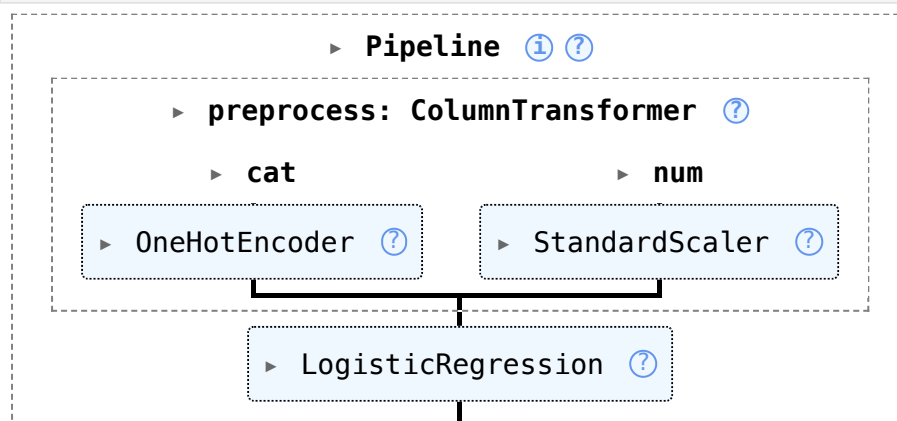
```
In [48]: #分割训练集与测试集

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, stratify=
```

```
In [49]: #开始训练

model.fit(X_train, y_train)
```

Out [49]:



4.1.3性能检查与前K名精确率(Precision@K)

```
In [50]: #查询预测概率
y_prob = model.predict_proba(X_test)[: , 1]
```

```
In [51]: #测试不同阈值对分类性能的影响

for thresholds in [0.5, 0.55, 0.6, 0.65, 0.7, 0.75]:
    y_pred_new = (y_prob >= thresholds).astype(int)
    print('Thresholds - ', thresholds)
    print(classification_report(y_test, y_pred_new))
    print(confusion_matrix(y_test, y_pred_new))
    print('=====')

thresholds = 0.65
y_pre_new = (y_prob >= thresholds).astype(int)
```

```
Thresholds - 0.5
precision    recall  f1-score   support

     0       0.95      0.87      0.91      457
     1       0.69      0.87      0.77      152

 accuracy
macro avg      0.82      0.87      0.84      609
weighted avg   0.89      0.87      0.88      609
```

```
[[399  58]
 [ 20 132]]
```

```
Thresholds - 0.55
precision    recall  f1-score   support

     0       0.94      0.89      0.92      457
     1       0.72      0.84      0.77      152

 accuracy
macro avg      0.83      0.86      0.84      609
weighted avg   0.89      0.88      0.88      609
```

```
[[407  50]
 [ 25 127]]
```

```
Thresholds - 0.6
precision    recall  f1-score   support

     0       0.93      0.91      0.92      457
     1       0.74      0.79      0.76      152

 accuracy
macro avg      0.83      0.85      0.84      609
weighted avg   0.88      0.88      0.88      609
```

```
[[415  42]
 [ 32 120]]
```

```
Thresholds - 0.65
precision    recall  f1-score   support

     0       0.92      0.92      0.92      457
     1       0.75      0.76      0.76      152

 accuracy
macro avg      0.84      0.84      0.84      609
weighted avg   0.88      0.88      0.88      609
```

```
[[419  38]
 [ 36 116]]
```

```
Thresholds - 0.7
precision    recall  f1-score   support

     0       0.91      0.92      0.92      457
     1       0.76      0.73      0.74      152

 accuracy
macro avg      0.84      0.83      0.83      609
```

weighted avg	0.87	0.88	0.87	609
--------------	------	------	------	-----

```
[[422 35]
 [ 41 111]]
```

```
=====
Thresholds - 0.75
              precision    recall  f1-score   support
```

0	0.91	0.93	0.92	457
1	0.77	0.71	0.74	152

accuracy			0.87	609
macro avg	0.84	0.82	0.83	609
weighted avg	0.87	0.87	0.87	609

```
[[424 33]
 [ 44 108]]
```

考虑到这是一个较高利润订单预测模型，为了避免销售资源与人力资源的浪费，可适当提高阈值，加强对模型的要求，提高模型预测的信心。同时考虑到整体的Accuracy也不能低，还要兼顾Precision-Recall trade off，我选择0.65作为分类阈值。一是因为该阈值对应Accuracy可达88%，说明模型整体性能良好；二是因为该任务属于少数派正例预测任务，此时比起关注Accuracy，更应该关注Precision；否则就会出现Accuracy过高，Precision过低这样的情况出现，模型整体性能不差，但实际没用，因为在这种情况下哪怕模型把所有订单都预测为较低利润订单，它依然可以取得75%的Accuracy。而该模型在0.65的阈值下，Precision可达75%，Recall也能达到76%，说明在模型预测/推荐的154个较高利润订单中，实际为较高利润订单为116个，并且在152个实际的较高利润订单中抓取到了116个。

另一方面，在与其他阈值的比较中，0.65也很有竞争力。例如假如现在将阈值升高到0.7，虽然可以减少3个误报订单，但会漏抓5个实际较高利润订单。考虑到较高利润订单对该公司贡献较大而且也为了避免资源的浪费，以多出3个误报的代价换取5个实际较高利润的订单的抓取将会是有意义的，划算的。而如果把阈值设为0.6的话，虽然可以多抓4个实际较高利润订单，但误报也会多出4个，同时Precision与Recall之间的差距也会进一步加大，即便0.65与0.6有相同的F1-Score。另一方面，我仍希望模型在推荐/预测较高利润订单的时候可以有更强的信心，所以综上所述，我将阈值设为0.65。

In [52]: `# 查询ROC与PR值，检查模型正确排序能力与少数类识别能力`

```
roc = roc_auc_score(y_test, y_prob)
pr = average_precision_score(y_test, y_prob)

print('ROC - ', roc)
print('PR - ', pr)
```

```
ROC - 0.9451946331912933
PR - 0.831021951700361
```

模型ROC值约为94.52%，说明模型在推荐排序上表现良好，有极高概率将较高利润订单排在普通订单之前，也就是说，有极高概率给较高利润订单一个较大的概率。PR值约为83.1%，说明模型抓取少数类的正确性较高，有较高概率抓取到实际的较高利润订单；这对我们的预测任务较为重要，因为在数据集中较高利润订单始终为少数，只占有所有数据的25%。

In [53]: # 查询模型在不同范围排名下的表现

```
result = X_test.copy()
result['true'] = y_test.values
result['prob'] = y_prob

for k in [10, 30, 50, 100]:
    topK = result.sort_values('prob', ascending=False).head(k)
    print(f'Precision@{k} - ', topK['true'].mean())
```

```
Precision@10 - 1.0
Precision@30 - 0.9333333333333333
Precision@50 - 0.92
Precision@100 - 0.82
```

可以看到，模型在前Top 10排名中抓取到实际较高利润订单的概率为100%，然后逐次下降，但即便将范围扩大到前Top 100依然有82%的概率抓取到实际较高利润订单。这意味着在模型推荐的前100个订单中，有82个订单为实际较高利润订单。

4.1.4数据泄露与无效变量

In [54]: #局限

```
featureName = model.named_steps['preprocess'].get_feature_names_out()
coeff = model.named_steps['clf'].coef_[0]
coeff_df = pd.DataFrame({'feature': featureName, 'coef': coeff}).sort_values('co
coeff_df.head(50)
```

Out [54]:

	feature	coef
24	num__Sales	3.029092
23	num__Unit Price	0.459944
14	cat__Product Name_Smartphone	0.313472
18	num__Quarter	0.272765
22	num__Quantity	0.140133
8	cat__Product Name_Headphones	0.112043
21	num__Weekday	0.072362
5	cat__Category_Electronics	0.068304
12	cat__Product Name_Mouse	0.025868
20	num__Day	0.025335
1	cat__Region_North	0.020892
11	cat__Product Name_Monitor	0.019005
7	cat__Product Name_Camera	0.013679
10	cat__Product Name_Laptop	0.004850
16	cat__Product Name_Tablet	-0.003092
17	num__Year	-0.014902
4	cat__Category_Accessories	-0.036449
2	cat__Region_South	-0.036724
0	cat__Region_East	-0.156355
9	cat__Product Name_Keyboard	-0.193366
15	cat__Product Name_Smartwatch	-0.260605
3	cat__Region_West	-0.267932
19	num__Month	-0.374757
6	cat__Category_Office	-0.471975
13	cat__Product Name_Printer	-0.471975

当人们查询各变量拟合后的系数时可以发现至少两个问题：一来是销售额实际上就等于单价乘销量，这其实就是我之前构造新的单价列的逆过程，所以在销售额与单价跟销量之间其实存在多重共线性，这会导致模型的方差较大，模型预测不稳定等问题；二来是在现实商业活动中，利润却是也与销售额存在强烈的相关性，例如高销售额的订单往往利润也很高，这就导致了数据泄露问题，进而使得模型并没有从数据中学到或者说提取到对预测较高利润订单真正有用的信息或者说影响因素。

另外，在现实生活中，在一个订单正式结束前，销量与单价依旧处于可变化的状态，例如只要交易还在继续，双方依旧可以讨价还价，进行一些购买数量与单价上的变更，所以比起保留销售额这一变量来说，保留单价与销量则更有意义。所以基于以上三个原因，我决定从数据集中剔除销售额这一变量，并保留单价与销量。

另一方面，在考虑时间相关的变量的时候，依旧存在着至少两个问题：一是年份与单纯的日子号数作为数值变量来说其实并没有对预测任务提供有效信息。二是季度变量与月份变量之间也存在着强烈的相关性，因为当你知道月份的时候，你也就知道了季度了。基于此，我决定从数据集中剔除年份，季度与日子号数这三个变量，保留月份与周内标识这两个变量。但是，考虑在实际商业活动中，购物这一行为也确实存在一定的周期性、季节性，所以我决定将月份与周内标识构造造成周期性变量，以此向模型提供更有用的信息。

4.2逻辑回归-第二次迭代

```
In [55]: data_lr = data_before_outlied.copy()
data_lr = data_lr.reset_index(drop = True)

# # 将Order data分解为 年, 年-月, 季度, 日, 周内标识, 五个新列, 并添加 单价 一个新列
data_lr['Order Date'] = pd.to_datetime(data_lr['Order Date'])

data_lr['Month'] = data_lr['Order Date'].dt.month.astype('int32')
data_lr['Weekday'] = data_lr['Order Date'].dt.dayofweek.astype('int32')
data_lr['Unit Price'] = data_lr['Sales']/data_lr['Quantity']

data_lr['Month_sin'] = np.sin(2 * np.pi * data_lr['Month'] / 12)
data_lr['Month_cos'] = np.cos(2 * np.pi * data_lr['Month'] / 12)

data_lr['Weekday_sin'] = np.sin(2 * np.pi * data_lr['Weekday'] / 7)
data_lr['Weekday_cos'] = np.cos(2 * np.pi * data_lr['Weekday'] / 7)

# # 选择必要的列并重新排列顺序
data_lr = data_lr[['Month_sin', 'Month_cos', 'Weekday_sin', 'Weekday_cos', 'Region']]

# # 转换产品名、地区与类别为分类类型
data_lr['Product Name'] = data_lr['Product Name'].astype('category')
data_lr['Region'] = data_lr['Region'].astype('category')
data_lr['Category'] = data_lr['Category'].astype('category')

data_lr.info()

<class 'pandas.DataFrame'>
RangeIndex: 3041 entries, 0 to 3040
Data columns (total 10 columns):
#   Column          Non-Null Count  Dtype
---  -
0   Month_sin       3041 non-null   float64
1   Month_cos       3041 non-null   float64
2   Weekday_sin     3041 non-null   float64
3   Weekday_cos     3041 non-null   float64
4   Region          3041 non-null   category
5   Category        3041 non-null   category
6   Product Name    3041 non-null   category
7   Quantity        3041 non-null   int64
8   Unit Price      3041 non-null   float64
9   Profit          3041 non-null   float64
dtypes: category(3), float64(6), int64(1)
memory usage: 175.5 KB
```

```
In [56]: #将利润大于前75%的订单标记为较高利润订单
```

```

bound = data_lr['Profit'].quantile(0.75)
y = (data_lr['Profit'] >= bound).astype(int)
X = data_lr[['Month_sin', 'Month_cos', 'Weekday_sin', 'Weekday_cos', 'Region', 'Cate

#编码各类数据并定义模型
cate_feature = ['Region', 'Category', 'Product Name']
num_feature = ['Month_sin', 'Month_cos', 'Weekday_sin', 'Weekday_cos', 'Quantity',

preprocess = ColumnTransformer(transformers=[('cat', OneHotEncoder(), cate_fea

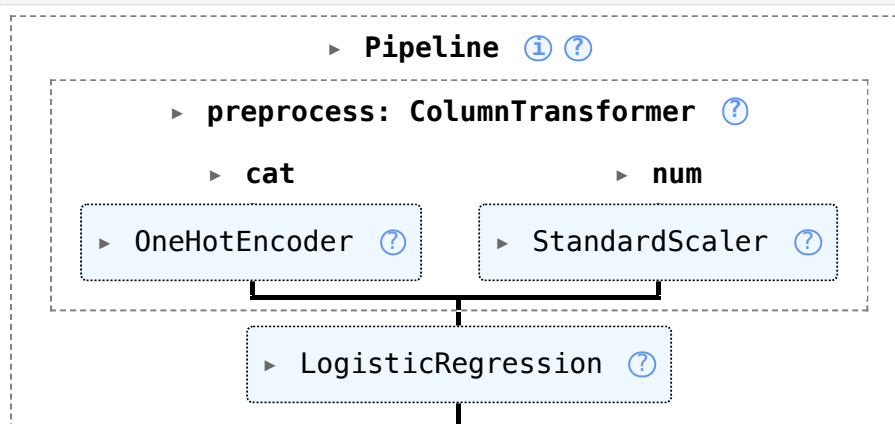
model = Pipeline(steps=[('preprocess', preprocess),
                        ('clf', LogisticRegression(max_iter=1000, class_weight='balance

#分割训练集与测试集
X_train, X_test, y_train, y_test = train_test_split(X,y,test_size=0.2,stratify=

#开始训练
model.fit(X_train, y_train)

```

Out [56]:



In [57]:

```

#查询预测概率
y_prob = model.predict_proba(X_test)[: , 1]

#测试不同阈值对分类性能的影响

for thresholds in [0.5, 0.55, 0.6, 0.65, 0.7, 0.75]:
    y_pred_new = (y_prob >= thresholds).astype(int)
    print('Thresholds - ', thresholds)
    print(classification_report(y_test, y_pred_new))
    print(confusion_matrix(y_test, y_pred_new))
    print('=====')

thresholds = 0.65
y_pre_new = (y_prob >= thresholds).astype(int)

```

```
Thresholds - 0.5
              precision    recall  f1-score   support

         0       0.95        0.83        0.89        457
         1       0.64        0.88        0.74        152

    accuracy                0.84        609
   macro avg       0.79        0.85        0.81        609
  weighted avg     0.87        0.84        0.85        609
```

```
[[381  76]
 [ 19 133]]
```

```
=====
Thresholds - 0.55
              precision    recall  f1-score   support

         0       0.94        0.87        0.90        457
         1       0.67        0.83        0.74        152

    accuracy                0.86        609
   macro avg       0.81        0.85        0.82        609
  weighted avg     0.87        0.86        0.86        609
```

```
[[396  61]
 [ 26 126]]
```

```
=====
Thresholds - 0.6
              precision    recall  f1-score   support

         0       0.93        0.88        0.90        457
         1       0.68        0.79        0.73        152

    accuracy                0.86        609
   macro avg       0.80        0.83        0.82        609
  weighted avg     0.87        0.86        0.86        609
```

```
[[401  56]
 [ 32 120]]
```

```
=====
Thresholds - 0.65
              precision    recall  f1-score   support

         0       0.92        0.90        0.91        457
         1       0.71        0.76        0.73        152

    accuracy                0.86        609
   macro avg       0.81        0.83        0.82        609
  weighted avg     0.87        0.86        0.86        609
```

```
[[410  47]
 [ 37 115]]
```

```
=====
Thresholds - 0.7
              precision    recall  f1-score   support

         0       0.90        0.91        0.91        457
         1       0.73        0.70        0.71        152

    accuracy                0.86        609
   macro avg       0.81        0.80        0.81        609
```

weighted avg	0.86	0.86	0.86	609
--------------	------	------	------	-----

```
[[417  40]
 [ 46 106]]
```

```
=====
Thresholds - 0.75
              precision    recall  f1-score   support
```

0	0.89	0.93	0.91	457
1	0.74	0.64	0.69	152

accuracy			0.86	609
macro avg	0.81	0.79	0.80	609
weighted avg	0.85	0.86	0.85	609

```
[[423  34]
 [ 54  98]]
```

基于同样的选择标准，我依旧选择0.65作为分类阈值；如果选择0.6，那Precision将下降到0.7以下；如果选择0.7，虽然能减少7个误报，但也会漏抓9个实际高利润订单，这样代价实在不划算。

如果人们对比前后两个模型的话，不难发现在相同0.65阈值下，新模型Recall保持不变，而Precision与F1-Score分别下降了4个百分点与3个百分点；预测为较高利润订单中的实际较高利润订单数量为115，比前一模型少了1个；误报数量为47，比前一模型多了9个，预测性能明显出现了下降。但好处就是后一模型具有更多的真实性与实际意义，因为我们考虑了现实生活的实际情况；同时也比前一模型的正确性要高，因为我们避免了数据泄露。

所以总的来说，该新模型依旧是一个构造正确的、预测有效的、实际有用的模型。

In [58]: # 查询ROC与PR值，检查模型正确排序能力与少数类识别能力

```
roc = roc_auc_score(y_test, y_prob)
pr = average_precision_score(y_test, y_prob)

print('ROC - ', roc)
print('PR - ', pr)
```

```
ROC - 0.9255009789243349
PR - 0.793851006266055
```

比起前一模型，不仅ROC下降了大约2个百分点，PR也下降了大约4个百分点。性能虽有下降，但仍在可接受范围内。

In [59]: # 查询模型在不同范围排名下的表现

```
result = X_test.copy()
result['true'] = y_test.values
result['prob'] = y_prob

for k in [10, 30, 50, 100]:
    topK = result.sort_values('prob', ascending=False).head(k)
    print(f'Precision@{k} - ', topK['true'].mean())
```

```
Precision@10 - 1.0  
Precision@30 - 0.9333333333333333  
Precision@50 - 0.92  
Precision@100 - 0.77
```

可以看到，前Top 10的排序中依旧100%正确的抓取到了较高利润订单，并且在前Top 30与前Top 50的排名中该模型性能与前一模型一模一样。然而前Top 100的排名性能就有一定的下降了，从82%降到了77%，也就是说，模型推荐的前100个较高利润订单中，实际较高利润订单的数量下降了5个；虽有不足，但仍可接受，仍具有实际意义。🎉

In [60]: #查看各变量对应系数

```
featureName = model.named_steps['preprocess'].get_feature_names_out()  
coeff = model.named_steps['clf'].coef_[0]  
coeff_df = pd.DataFrame({'feature': featureName, 'coef': coeff}).sort_values('co  
coeff_df.head(50)
```

Out [60]:

	feature	coef
22	num__Unit Price	3.833983
21	num__Quantity	2.756930
8	cat__Product Name_Headphones	0.281110
11	cat__Product Name_Monitor	0.252376
14	cat__Product Name_Smartphone	0.240599
5	cat__Category_Electronics	0.202071
7	cat__Product Name_Camera	0.168157
4	cat__Category_Accessories	0.111266
1	cat__Region_North	0.085412
2	cat__Region_South	0.058726
16	cat__Product Name_Tablet	0.052744
17	num__Month_sin	0.046398
18	num__Month_cos	0.024877
0	cat__Region_East	-0.019233
19	num__Weekday_sin	-0.029829
15	cat__Product Name_Smartwatch	-0.075518
12	cat__Product Name_Mouse	-0.103235
3	cat__Region_West	-0.121180
20	num__Weekday_cos	-0.151723
10	cat__Product Name_Laptop	-0.183911
6	cat__Category_Office	-0.309613
13	cat__Product Name_Printer	-0.309613
9	cat__Product Name_Keyboard	-0.318985

与前一模型的系数做一个比较的话，人们可以发现，前一个模型的权重都集中在销售额这一变量上；而当我们剔除这一变量后，单价与销量之前被它吸收的权重就明显上升了。同时还可以发现，月份与利润之间确实存在一定影响，但从数值方面来讲，影响较小，甚至周内标识的对应权重都下降成了负数。

4.3随机森林

为了深入探索时间类型变量对利润的影响，现对同样的预测任务构造随机森林模型；并且在完成训练后同样对分类阈值进行筛查并查询预测性能与排名性能，同时查看各变量对应系数。

```
In [61]: data_lr = data_before_outlied.copy()
data_lr = data_lr.reset_index(drop = True)

# # 将Order data分解为 年, 年-月, 季度, 日, 周内标识, 五个新列, 并添加 单价 一个新列
```

```

data_lr['Order Date'] = pd.to_datetime(data_lr['Order Date'])

data_lr['Month'] = data_lr['Order Date'].dt.month.astype('int32')
data_lr['Weekday'] = data_lr['Order Date'].dt.dayofweek.astype('int32')
data_lr['Unit Price'] = data_lr['Sales']/data_lr['Quantity']

data_lr['Month_sin'] = np.sin(2 * np.pi * data_lr['Month'] / 12)
data_lr['Month_cos'] = np.cos(2 * np.pi * data_lr['Month'] / 12)

data_lr['Weekday_sin'] = np.sin(2 * np.pi * data_lr['Weekday'] / 7)
data_lr['Weekday_cos'] = np.cos(2 * np.pi * data_lr['Weekday'] / 7)

# 选择必要的列并重新排列顺序
data_lr = data_lr[['Month_sin', 'Month_cos', 'Weekday_sin', 'Weekday_cos', 'Region']]

# 转换产品名、地区与类别为分类类型
data_lr['Product Name'] = data_lr['Product Name'].astype('category')
data_lr['Region'] = data_lr['Region'].astype('category')
data_lr['Category'] = data_lr['Category'].astype('category')

# 将利润大于前75%的订单标记为较高利润订单
bound = data_lr['Profit'].quantile(0.75)
y = (data_lr['Profit'] >= bound).astype(int)
X = data_lr[['Month_sin', 'Month_cos', 'Weekday_sin', 'Weekday_cos', 'Region', 'Category']]

# 数据集分割
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, stratify=y)

# 编码各类数据并定义模型
cate_feature = ['Region', 'Category', 'Product Name']
num_feature = ['Month_sin', 'Month_cos', 'Weekday_sin', 'Weekday_cos', 'Quantity']

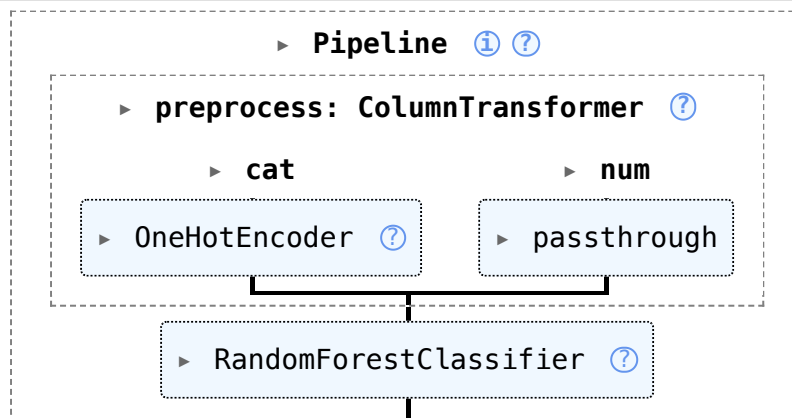
# 预处理各列数据
preprocess_rf = ColumnTransformer(transformers=[('cat', OneHotEncoder(), cate_feature)],
                                  remainder='passthrough')

model_rf = Pipeline(
    steps=[('preprocess', preprocess_rf), ('clf', RandomForestClassifier(
        n_estimators=300, max_depth=None,
        min_samples_leaf=5, class_weight='balanced', random_state=42))])

model_rf.fit(X_train, y_train)

```

Out [61]:



In [62]:

```

# 预测为高利润订单的概率
y_prob_rf = model_rf.predict_proba(X_test)[:, 1]

for thresholds in [0.5, 0.55, 0.6, 0.65, 0.7]:

```

```
y_pred_t = (y_prob_rf >= thresholds).astype(int)
print('thresholds - ', thresholds)
print(classification_report(y_test, y_pred_t))
print(confusion_matrix(y_test, y_pred_t))
print('=====')

threshold = 0.55
y_pred_t = (y_prob_rf >= threshold).astype(int)
```

```
thresholds - 0.5
precision    recall  f1-score   support

     0       0.95      0.86      0.90      457
     1       0.67      0.86      0.75      152

accuracy
macro avg      0.81      0.86      0.83      609
weighted avg    0.88      0.86      0.87      609
```

```
[[394 63]
 [ 22 130]]
```

```
thresholds - 0.55
precision    recall  f1-score   support

     0       0.93      0.88      0.91      457
     1       0.70      0.80      0.74      152

accuracy
macro avg      0.81      0.84      0.82      609
weighted avg    0.87      0.86      0.87      609
```

```
[[404 53]
 [ 31 121]]
```

```
thresholds - 0.6
precision    recall  f1-score   support

     0       0.91      0.90      0.91      457
     1       0.72      0.74      0.73      152

accuracy
macro avg      0.81      0.82      0.82      609
weighted avg    0.86      0.86      0.86      609
```

```
[[413 44]
 [ 40 112]]
```

```
thresholds - 0.65
precision    recall  f1-score   support

     0       0.90      0.93      0.91      457
     1       0.76      0.69      0.72      152

accuracy
macro avg      0.83      0.81      0.82      609
weighted avg    0.86      0.87      0.86      609
```

```
[[423 34]
 [ 47 105]]
```

```
thresholds - 0.7
precision    recall  f1-score   support

     0       0.88      0.94      0.91      457
     1       0.78      0.62      0.69      152

accuracy
macro avg      0.83      0.78      0.80      609
```

weighted avg	0.86	0.86	0.86	609
--------------	------	------	------	-----

```
[[430 27]
 [ 57 95]]
```

基于相同的标准，我将阈值选为0.55。在这一个阈值下，Accuracy表现为86%，Precision与Recall分别为70%，80%，均在70%到80%范围间。如果选择0.5，则Precision将降到70%以下；如果选择0.6，将多增加9个实际较高利润订单漏报，这样的代价显然是无法接受的。

In [63]: `#查询ROC与PR值以及模型在排名方面的表现`

```
roc_rf = roc_auc_score(y_test, y_prob_rf)
pr_rf = average_precision_score(y_test, y_prob_rf)

print('ROC - ', roc_rf)
print('PR - ', pr_rf)

result_rf = X_test.copy()
result_rf['true'] = y_test.values
result_rf['prob'] = y_prob_rf

for k in [10, 30, 50, 100]:
    topK = result_rf.sort_values('prob', ascending=False).head(k)
    print(f'Precision@{k} - ', topK['true'].mean())
```

```
ROC - 0.935693308764252
PR - 0.7964569912334365
Precision@10 - 1.0
Precision@30 - 0.8666666666666667
Precision@50 - 0.86
Precision@100 - 0.78
```

ROC较上一个逻辑回归模型上升了约1个百分点，PR变化未超过1个百分点，说明随机森林这一新模型整体预测能力与正确抓取实际较高利润能力并未发生太大变化。前Top 10排名性能依旧为100%，前Top30与前Top50排名性能依次降低7个百分点与6个百分点，然而前Top 100排名性能则上升一个百分点。排名性能整体略有下降，但仍可接受，其中前Top 100排名性能略有上升。该模型排名能力仍具有实际意义。

In [64]: `#查看各变量对应系数重要性`

```
feature_names = model_rf.named_steps['preprocess'].get_feature_names_out()

importances = model_rf.named_steps['clf'].feature_importances_

importance_df = pd.DataFrame({
    'feature': feature_names,
    'importance': importances
}).sort_values('importance', ascending=False)

importance_df.head(50)
```

Out [64]:

	feature	importance
22	num__Unit Price	0.413059
21	num__Quantity	0.385847
18	num__Month_cos	0.026929
17	num__Month_sin	0.026231
19	num__Weekday_sin	0.025450
20	num__Weekday_cos	0.020494
14	cat__Product Name_Smartphone	0.014052
9	cat__Product Name_Keyboard	0.009698
5	cat__Category_Electronics	0.007843
10	cat__Product Name_Laptop	0.007711
3	cat__Region_West	0.007624
1	cat__Region_North	0.007130
0	cat__Region_East	0.007056
2	cat__Region_South	0.006541
4	cat__Category_Accessories	0.006020
16	cat__Product Name_Tablet	0.005441
12	cat__Product Name_Mouse	0.004560
11	cat__Product Name_Monitor	0.004501
15	cat__Product Name_Smartwatch	0.004486
7	cat__Product Name_Camera	0.004123
8	cat__Product Name_Headphones	0.002358
13	cat__Product Name_Printer	0.001591
6	cat__Category_Office	0.001257

可以看出，单价与销量的重要性仍居前两位，月份与周内标识与逻辑回归第二次迭代的模型相比，重要性则显著提高我就说嘛，这说明较高利润的订单确实与时间的周期性、季节性存在一定相关性。这也与实际现实情况相符，例如在中国每年的1-3月正值春节，属于消费旺季，大额订单在这段时间里出现的概率也较大；而在国外的11-1月则由于黑色星期五以及圣诞节、元旦等因素，也会出现周期性的购物潮，进而促使大额订单的出现。而对周内标识来说，依然符合实际趋势，例如一周内人们也往往趋向于周四或者周五以后再去购物。

5.结论、感悟与展望

该项目从时间、空间、产品等不同维度对该企业三年内的销售数据进行了分析总结，成功发现了该企业在不同时期与不同区域的销售趋势与盈利模式，识别出了该企业增长质量较差的后续风险；同时通过对该企业所营产品的分析，挖掘出了该公司产品对公司发展的潜在优势；另外还对异常大额订单进行了额外分析，为该公司未来的利润增长探索了潜在的商业机会。最后通过机器学习技术，向该公司交付了不同的较高利润订单预测模型，用以辅助公司日常运营与动态销售策略的决策，为该企业的高质量发展提供了新的动力。

从该项目中也可以看到，一个企业的发展不仅与他本身的产品结构、经营策略有关，而且也会受时间的周期性的、季节性的影响，同时与区域的不同也有一定的关系；这就说明了企业的经营与发展不该过度依赖特定的时间地点、特定的产品与特定的销售策略，而是应该具体问题具体分析，根据不同的时期区域、不同的产品市场，来动态的构建发展策略，进而有效的规避发展风险，实现企业高质量的发展。

然而，该项目也存在一定的局限与不足。从商业的角度来说，例如产品具体型号与品牌信息的缺失，使得产品的总结与大额订单的分析无法深入到具体产品这一层面，导致了得出的结论不够具体而且也无法构造对潜在的较高利润的具体产品订单的预测模型。另一方面，从技术的角度来讲，由于较高利润订单数据的稀缺，预测模型的训练也变得困难，这也直接导致了模型预测性能以及排名性能的瓶颈。在以后的工作中，基于足够多的、质量足够好的数据，还可以尝试从更多角度、更广层面对数据进行处理与分析，并由此向企业提供更有价值与意义的结论与意见；同时基于大量的数据和不同的机器学习算法，还可以向企业交付性能更好，鲁棒性更强的商业模型；多角度、全方位、跨领域得为企业进步提供优质的动力，共同促进企业的高质量发展。

声明：由于本人并非商业科学学生，该项目中涉及到的商业概念大半为自学；若有理解不到位甚至出错的地方，望各位可以直接指出；谢谢。:]